Support de cours

Services cloud (Google Cloud)

Classes : B3-ASI

Enseignant : AGOUTOCO-POKO Caleb Benaja
WhatsApp (00228) 92227441, Tel : 99990031

Année universitaire : 2025/2026

MODULE 4 : Développement et Déploiement sur Google Cloud

Introduction

Après avoir découvert les concepts fondamentaux du cloud, l'infrastructure Google Cloud et la gestion des identités et des accès (IAM), il est essentiel de comprendre comment développer, tester et déployer des applications dans un environnement cloud moderne.

Google Cloud fournit une plateforme complète permettant aux développeurs et aux équipes informatiques de créer, déployer, exécuter et gérer des applications de manière efficace, sécurisée et évolutive. Grâce à ses nombreux services, les organisations peuvent accélérer le développement logiciel, réduire les coûts d'infrastructure et améliorer la disponibilité de leurs applications.

Le développement sur Google Cloud repose sur plusieurs approches, notamment les machines virtuelles, les conteneurs, les architectures sans serveur (serverless) et les plateformes de développement intégrées. Les développeurs peuvent ainsi choisir la solution la plus adaptée aux besoins de leur application.

Le déploiement des applications dans Google Cloud s'appuie également sur des outils modernes d'automatisation tels que l'intégration continue et le déploiement continu (CI/CD), permettant de livrer rapidement de nouvelles fonctionnalités tout en garantissant la qualité et la stabilité des services.

Les principaux services de développement et de déploiement proposés par Google Cloud, notamment :

- **Google Compute Engine** pour l'hébergement sur machines virtuelles ;
- **Google App Engine** pour le déploiement d'applications sans gestion d'infrastructure ;
- **Google Kubernetes Engine (GKE)** pour l'orchestration des conteneurs ;
- **Cloud Run** pour l'exécution d'applications conteneurisées en mode serverless ;
- **Cloud Build** pour l'automatisation des processus de construction et de déploiement ;
- **Artifact Registry** pour le stockage et la gestion des images de conteneurs et des artefacts logiciels

Google Compute Engine
Est le service de machines virtuelles de Google Cloud. Il permet de créer des serveurs virtuels pour héberger des sites web, applications métier, bases de données, services réseau ou environnements de test. C'est l'un des services les plus utilisés par les administrateurs systèmes, ingénieurs réseaux et développeurs sur Google Cloud.

2. Déploiement avec App Engine (GAE)

Définition

Google App Engine (GAE) est un service **PaaS (Platform as a Service)** de Google Cloud qui permet de développer, déployer et exécuter des applications web et mobiles sans avoir à gérer les serveurs, les systèmes d'exploitation ou l'infrastructure sous-jacente.

- La disponibilité des services.
- La mise à l'échelle (scaling) ;
- Les mises à jour ;
- La sécurité ;
- La disponibilité des services ;

Qu’est-ce qu’App Engine ?

Le **déploiement avec App Engine** consiste à publier une application sur la plateforme Google Cloud afin qu'elle soit accessible aux utilisateurs via Internet.

Contrairement à Google Compute Engine où l'on doit créer et administrer une machine virtuelle, App Engine permet de déployer directement le code de l'application.

- **PaaS** de Google Cloud → vous déployez le code, GCP gère tout le reste

- Supporte plusieurs langages : **Python, Node.js, Java, Go...**
- Deux environnements :
    - **Standard** : rapide à démarrer, ressources limitées
    - **Flexible** : plus personnalisable (Docker, plus de puissance)

Exemple : Déploiement d'une app Python Flask

```
app.yaml :
runtime: python310
entrypoint: gunicorn -b :$PORT main:app
```

```
# main.py
from flask import Flask
app = Flask(__name__)

@app.route('/')
def home():
    return 'Bienvenue sur App Engine !'
```

```
# Déploiement
gcloud app deploy
```

2. Conteneurisation avec Docker

Définition

La **conteneurisation** est une technologie qui permet d'emballer une application ainsi que toutes ses dépendances (bibliothèques, configurations, fichiers système, etc.) dans un **conteneur** léger et portable.

Docker est la plateforme de conteneurisation la plus populaire. Elle permet de créer, déployer et exécuter des applications dans des conteneurs de manière rapide et cohérente, quel que soit l'environnement (ordinateur personnel, serveur ou cloud).

Qu'est-ce qu'un conteneur ?

- Environnement léger et portable pour exécuter des applications
- Contient le code + dépendances + système

Exemple : Dockerfile pour une app Node.js

```
FROM node:18-alpine
WORKDIR /app
COPY . .
RUN npm install
CMD ["node", "index.js"]
```

```
docker build -t my-node-app .
docker run -p 8080:8080 my-node-app
```

3. Déploiement avec Google Kubernetes Engine (GKE)

Définition

Google Kubernetes Engine (GKE) est un service managé de Google Cloud qui permet de déployer, gérer et faire évoluer des applications conteneurisées à l'aide de **Kubernetes**.

Kubernetes est une plateforme d'orchestration de conteneurs qui automatise :

- Le déploiement des applications ;
- La gestion des conteneurs ;
- La mise à l'échelle (scaling) ;
- La haute disponibilité ;
- La surveillance et la reprise après incident.

Avec GKE, Google Cloud gère l'infrastructure Kubernetes, ce qui simplifie considérablement l'administration.

Qu’est-ce que Kubernetes ?

- Système d’orchestration de conteneurs
- Permet le **scalabilité**, **load balancing**, **auto-recovery**

Exemple de déploiement d’un conteneur

```yaml
# deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: hello-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: hello
  template:
    metadata:
      labels:
        app: hello
    spec:
      containers:
      - name: hello
        image: gcr.io/mon-projet/hello-app
        ports:
        - containerPort: 8080
```

```
kubectl apply -f deployment.yaml
```

4. CI/CD avec Cloud Build & Cloud Source Repositories

Introduction

Dans le développement moderne, les entreprises publient régulièrement de nouvelles fonctionnalités, corrigent des bugs et améliorent leurs applications. Pour accélérer ce processus tout en réduisant les erreurs humaines, elles utilisent les pratiques **CI/CD (Continuous Integration / Continuous Deployment)**.

Google Cloud fournit plusieurs services permettant de mettre en œuvre ces pratiques, notamment :

- **Cloud Source Repositories** : hébergement du code source.
- **Cloud Build** : automatisation de la construction, des tests et du déploiement des applications.

Qu'est-ce que le CI/CD ?

CI : Continuous Integration (Intégration Continue)

L'intégration continue consiste à fusionner régulièrement les modifications de code dans un dépôt central.

À chaque modification :

- Le code est compilé ;
- Les tests sont exécutés automatiquement ;
- Les erreurs sont détectées rapidement.

Objectifs

- Détecter rapidement les bugs ;

- Améliorer la qualité du code ;
- Faciliter le travail collaboratif.

CD : Continuous Deployment / Continuous Delivery

Continuous Delivery

L'application est automatiquement préparée pour être déployée.

Le déploiement en production nécessite une validation humaine.

Continuous Deployment

Après validation des tests, le déploiement est automatique sans intervention humaine.

Cloud Source Repositories

Définition

Cloud Source Repositories est un service Git de Google Cloud permettant de stocker et gérer le code source des applications.

Fonctionnalités :

- Gestion du code source ;
- Historique des versions ;
- Collaboration entre développeurs ;
- Intégration avec Cloud Build.

Cloud Build

Définition

Cloud Build est un service d'intégration et de déploiement continu de Google Cloud.

Il automatise :

- La compilation ;
- Les tests ;
- La création d'images Docker ;
- Le déploiement des applications.

Exemple cloudbuild.yaml

```yaml
steps:
- name: 'gcr.io/cloud-builders/docker'
  args: ['build', '-t', 'gcr.io/$PROJECT_ID/my-app', '.']
- name: 'gcr.io/cloud-builders/docker'
  args: ['push', 'gcr.io/$PROJECT_ID/my-app']
images:
- 'gcr.io/$PROJECT_ID/my-app'
```

TP – Déploiement d'une application web sur Google Cloud

Matériel nécessaire

- Compte GCP avec un projet activé
- Cloud SDK (gcloud) installé
- Git + éditeur de code local
- Langage au choix :
  - Node.js
  - Python (Flask)

Partie 1 – Déploiement avec App Engine Standard

Étape 1 : Créer un projet de base

Choisissez un langage (ici : exemple Flask)

- **main.py**

```python
from flask import Flask
app = Flask(__name__)

@app.route('/')
def hello():
    return "Bonjour depuis App Engine !"

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=8080)
```

- requirements.txt

```
Flask==2.3.2
```

- app.yaml

```
runtime: python310
entrypoint: gunicorn -b :$PORT main:app
```

Étape 2 : Déploiement

```
gcloud app create --region=europe-west1
gcloud app deploy
```

Attendez la fin du déploiement, puis ouvrez l'URL proposée.

Partie 2 – Conteneurisation avec Docker

Étape 3 : Ajouter un Dockerfile

```
# Dockerfile
FROM python:3.10-slim
WORKDIR /app
COPY . .
RUN pip install -r requirements.txt
CMD ["gunicorn", "-b", ":8080", "main:app"]
```

Étape 4 : Construire et tester en local

```
docker build -t hello-flask .
docker run -p 8080:8080 hello-flask
```

Vérifiez que l'app tourne sur http://localhost:8080.

Partie 3 – Déploiement sur Google Cloud avec Cloud Build

Étape 5 : Créer un dépôt Git

```
gcloud source repos create demo-app
git remote add google https://source.developers.google.com/p/YOUR_PROJECT_ID/r/demo-app
git push google main
```

Étape 6 : Ajouter un fichier cloudbuild.yaml

- name: 'gcr.io/cloud-builders/docker'
  args: ['build', '-t', 'gcr.io/$PROJECT_ID/hello-flask', '.']
- name: 'gcr.io/cloud-builders/docker'
  args: ['push', 'gcr.io/$PROJECT_ID/hello-flask']
- name: 'gcr.io/cloud-builders/gcloud'
  args: ['run', 'deploy', 'hello-flask', '--image', 'gcr.io/$PROJECT_ID/hello-flask', '--region', 'europe-west1', '--platform', 'managed', '--allow-unauthenticated']
images:
- 'gcr.io/$PROJECT_ID/hello-flask'

Étape 7 : Déclenchement automatique du pipeline

1. Menu GCP > **Cloud Build > Déclencheurs**
2. Créez un **déclencheur basé sur Git**
   - Branche : main
   - cloudbuild.yaml
3. Faites un git push

Cloud Build va :

- Construire l'image Docker
- La pousser dans Artifact Registry
- Déployer sur **Cloud Run**

Résultat attendu

- Une app web disponible via App Engine **(Partie 1)**
- Un conteneur Docker local fonctionnel **(Partie 2)**
- Un déploiement CI/CD automatisé via **Cloud Build (Partie 3)**